

Ķekavas vidusskola

Budžets

Programmēšana II

Projekta darba autore: Tīna Ozola 12.a klase
Projekta darba vadītāja: Maiga Pīgita

2025./2026.m.g.

Mērķis:

Šīs programmas mērķis ir palīdzēt lietotājiem ērti sekot līdzi saviem ienākumiem un izdevumiem. Lietotājs var pievienot finanšu ierakstus, redzēt kopējo bilanci un analizēt savus tēriņus.

Programma ļauj:

- Uzrakstīt ienākumus un izdevumus
- Aprēķināt bilanci
- Saglabāt datus failā
- Pārskatīt un filtrēt ierakstus

Kā lietot programmu:

Budžeta plānotājs

Dark mode

Sākums

Bilance

Ienākumi
0.00 €

Izdevumi
0.00 €

Bilance
0.00 €

Pievienot ierakstu

mm/dd/yyyy

Summa

Apraksts

Ienākums

Pievienot

Filtrēšana

Visi

Filtrēt

Ieraksti

Nav neviena ieraksta.

1. attēls. Budžet aplikācija. Autores veidots

Pievienot ierakstu:

- Ievadi datumu
- Ievadi summu
- Ievadi aprakstu
- Izvēlies tipu (izdevums vai ienākums)
- Nospied "Pievienot"

Budžeta plānotājs

Dark mode

Sākums

Bilance

Ienākumi

12.40 €

Izdevumi

5.30 €

Bilance

7.10 €

Pievienot ierakstu

mm/dd/yyyy

Summa

Apraksts

Ienākums

Pievienot

Filtrēšana

Visi

Filtrēt

Ieraksti

Datums	Tips	Apraksts	Summa	Darbība
2026-01-14	Ienākums	Gift	12.40 €	Dzēst
2026-03-19	Izdevums	lunch	5.30 €	Dzēst

2. attēls. Aplikācijas ieraksti. Autores veidots

Lai filtrētu ierakstus:

- Izvēlies filtru (visi /ienākumi/ izdevumi)
- Nospied pogu "Filtrēt"

Budžeta plānotājs Dark mode

Sākums Balance

Ienākumi
12.40 €

Izdevumi
5.30 €

Balance
7.10 €

Pievienot ierakstu

mm/dd/yyyy
Summa
Apraksts
Ienākums
Pievienot

Filtrēšana

Tikai ienākumi
Filtrēt

Ieraksti

Datums	Tips	Apraksts	Summa	Darbība
2026-01-14	Ienākums	Gift	12.40 €	Dzēst

3. attēls. Filtrēšanas sistēma. Autores veidots

Lai dzēstu ierakstu:

- Nospied pogu “Dzēst”, kas atrodas ieraksta labajā pusē.

Lai mainītu dizainu:

- Nospied “Dark/Light mode” pogu, un dizains vai nu paliks gaišāks, vai tumšāks.

Budžeta plānotājs Light mode

Sākums Balance

Ienākumi
0.00 €

Izdevumi
5.30 €

Balance
-5.30 €

Pievienot ierakstu

mm/dd/yyyy
Summa
Apraksts
Ienākums
Pievienot

Filtrēšana

Visi
Filtrēt

Ieraksti

Datums	Tips	Apraksts	Summa	Darbība
2026-03-19	Izdevums	lunch	5.30 €	Dzēst

4. attēls. “Dark mode”. Autores veidots

Budžeta plānotājs Dark mode

Sākums Balance

Ienākumi
0.00 €

Izdevumi
5.30 €

Balance
-5.30 €

Pievienot ierakstu

mm/dd/yyyy
Summa
Apraksts
Ienākums
Pievienot

Filtrēšana

Visi
Filtrēt

Ieraksti

Datums	Tips	Apraksts	Summa	Darbība
2026-03-19	Izdevums	lunch	5.30 €	Dzēst

5. attēls. “Light mode”. Autores veidots

Koda apraksts (galvenās funkcijas):

❖ ieladet_datus()

```
def ieladet_datus():
    """Ielādē datus no CSV faila, ja tas eksistē."""
    global dati
    dati = []

    if os.path.exists(CSV_FAILS):
        with open(CSV_FAILS, mode="r", newline="", encoding="utf-8") as f:
            lasitajs = csv.DictReader(f)
            for rinda in lasitajs:
                dati.append({
                    "id": int(rinda["id"]),
                    "datums": rinda["datums"],
                    "tips": rinda["tips"],
                    "summa": float(rinda["summa"]),
                    "apraksts": rinda["apraksts"]
                })
```

6. attēls. Funkcija, kas ielādē datus. Autores veidots

Ielādē datus no CSV faila (dati.csv), pārbauda, vai fails eksistē (os.path.exists), nolasa datus ar csv.DictReader, saglabā ierakstus sarakstā dati.

❖ saglabat_datus()

```
def saglabat_datus():
    """Saglabā visus datus CSV failā."""
    with open(CSV_FAILS, mode="w", newline="", encoding="utf-8") as f:
        lauki = ["id", "datums", "tips", "summa", "apraksts"]
        rakstitajs = csv.DictWriter(f, fieldnames=lauki)
        rakstitajs.writeheader()
        rakstitajs.writerows(dati)
```

7. attēls. Funkcija, kas saglabā datus. Autores veidots

Saglabā visus ierakstus CSV failā, ieraksta galveni (id, datums, tips, summa, apraksts), saglabā visus datus no saraksta dati. Nodrošina datu saglabāšanu.

❖ nakamais_id()

```
def nakamais_id():
    """Atgriež nākamo brīvo ID."""
    if not dati:
        return 1
    return max(ieraksts["id"] for ieraksts in dati) + 1
```

9. attēls. Funkcija, kas atgriež nākamo unikālo ID. Autores veidots

Atgriež nākamo unikālo ID, ja ieraksts tukšs - atgriež 1, citādi - atrod lielāko esošo ID un pieskaita +1. Katram ierakstam ir unikāls identifikators.

❖ aprekinat_kopsavilkumu()

```
def aprekinat_kopsavilkumu(ieraksti):
    """Aprēķina ienākumus, izdevumus un bilanci."""
    ienakumi = sum(i["summa"] for i in ieraksti if i["tips"] == "Ienākums")
    izdevumi = sum(i["summa"] for i in ieraksti if i["tips"] == "Izdevums")
    balance = ienakumi - izdevumi
    return ienakumi, izdevumi, balance
```

10. attēls. Funkcija, kas aprēķina kopsavilkumu. Autores veidots

Aprēķina finanšu kopsavilkumu, saskaita ienākumus un izdevumus, aprēķina bilanci. Izmanto gan galvenajā lapā, gan balances lapā.

❖ index()

```
@app.route("/")
def index():
    filters = request.args.get("filters", "Visi")

    if filters == "Ienākums":
        filtretie = [i for i in dati if i["tips"] == "Ienākums"]
    elif filters == "Izdevums":
        filtretie = [i for i in dati if i["tips"] == "Izdevums"]
    else:
        filtretie = dati

    ienakumi, izdevumi, balance = aprekinat_kopsavilkumu(dati)

    return render_template(
        "index.html",
        dati=filtretie,
        filters=filters,
        ienakumi=ienakumi,
        izdevumi=izdevumi,
        balance=balance,
        kluda=""
    )
```

11. attēls. Funkcija, kas atbild par datu attēlošanu. Autores veidots

Galvenā lapa, parāda visus ierakstus, ļauj filtrēt (visi / tikai ienākumi / tikai izdevumi), aprēķina kopsavilkumu. Atbild par datu attēlošanu lietotājam.

❖ pievienot()

```
@app.route("/pievienot", methods=["POST"])
def pievienot():
    tips = request.form.get("tips", "").strip()
    summa_teksts = request.form.get("summa", "").strip()
    apraksts = request.form.get("apraksts", "").strip()
    datums = request.form.get("datums", "").strip()

    kluda = ""

    if tips not in ["Ienākums", "Izdevums"]:
        kluda = "Nepareizs ieraksta tips."
    elif not summa_teksts:
        kluda = "Lūdzu ievadi summu."
    elif not apraksts:
        kluda = "Lūdzu ievadi aprakstu."
    elif not datums:
        kluda = "Lūdzu izvēlies datumu."
    else:
        try:
            summa = float(summa_teksts)
            if summa <= 0:
                kluda = "Summai jābūt lielākai par 0."
        except ValueError:
            kluda = "Summai jābūt skaitlim."
```

12. attēls. Funkcija, kas atbild par datu pievienošanu (1). Autores veidots

```
if kluda:
    filters = request.args.get("filters", "Visi")
    filtetie = dati
    ienakumi, izdevumi, balance = aprekinat_kopsavilkumu(dati)
    return render_template(
        "index.html",
        dati=filtetie,
        filters=filters,
        ienakumi=ienakumi,
        izdevumi=izdevumi,
        balance=balance,
        kluda=kluda
    )

ieraksts = {
    "id": nakamais_id(),
    "datums": datums,
    "tips": tips,
    "summa": summa,
    "apraksts": apraksts
}

dati.append(ieraksts)
saglabat_datus()
return redirect(url_for("index"))
```

13. attēls. Funkcija, kas atbild par datu pievienošanu (2). Autores veidots

Pievieno jaunu ierakstu. Nolasa datus no formas (request.form), validē vai laiki nav tukši, vai summa ir skaitlis (float). Izveido jaunu ierakstu (dictionary), pievieno sarakstam (dati), saglabā CSV failā.

Atbild par jaunu datu pievienošanu.

❖ dzest()

```
@app.route("/dzest/<int:ieraksta_id>", methods=["POST"])
def dzest(ieraksta_id):
    global dati
    dati = [i for i in dati if i["id"] != ieraksta_id]
    saglabat_datus()
    return redirect(url_for("index"))
```

14. attēls. Funkcija, kas atbild par datu dzēšanu. Autores veidots

Dzēš ierakstu. Atros ierakstu pēc ID, izņem to no saraksta, saglabā izmaiņas CSV failā. Ļauj lietotājam dzēst nevajadzīgus ierakstus.

❖ balance_lapa()

```
@app.route("/balance")
def balance_lapa():
    ienakumi, izdevumi, balance = aprekinat_kopsavilkumu(dati)
    return render_template(
        "balance.html",
        ienakumi=ienakumi,
        izdevumi=izdevumi,
        balance=balance
    )

if __name__ == "__main__":
    ieladet_datus()
    app.run(debug=True)
```

14. attēls. Funkcija, kas parāda bilances lapu. Autores veidots

Parāda bilances lapu. Aprēķina ienākumus, izdevumus un bilanci, attēlo tos atsevišķā lapā.

Dod pārskatāmu finanšu kopsavilkumu.

Secinājumi:

Bija viegli strādāt ar ChatGPT.com, lai izveidotu pašu programmu, grūtākā daļa bija programmas pārbaude un dokumenta veidošana. Programma strādā tieši, kā bija plānots.